

spec.md

CSE 110 Teams #1-5 Sp 26 Project

Overview

With our team formed and enough practice with GenAI, in both a code focused and design-focused manner, we are ready to take on a project utilizing modern Agile methodologies.

While the project deliverable in the form of working quality software is quite important, students are strongly reminded that the process we undertake is the actual focus in our academic exploration of SWE. **A repeatable and observable quality focused process that ends up producing less feature rich software will get a higher grade than a large feature rich piece of software done opaquely or with less process.**

TL;DR - For a course, **process >> product**. Beyond a course, that may not hold.

Agile and Process Requirements

Your team will be required to

- Perform a documented sprint planning meeting before starting work each sprint and capture the information in your repository (GitHub)
- Hold stand-up meetings virtually (Slack) and/or in-person at least 3 times a week and capture the information in your repository (GitHub)
- Tasks are captured in an issue tracker (GitHub Issues) and work happens and is documented in the issue tracker
- Perform a sprint review and retrospective at least two times in the remaining time of the quarter. Information from the retrospective must be captured and there must be evidence of its incorporation
- Meet with your TA weekly and capture the meeting information (GitHub)
- Meet with the Prof once before the conclusion of the quarter

Some meetings may not include all team members. For example, TA or Prof meetings may be attended by two or three people, but planning and retrospective meetings must be attended by all.

Repository, Process, and Tooling Requirements

- All work including planning, meeting information, tests, and documentation must be captured in GitHub incrementally
- GenAI may be used, and if used, must be exposed and discussed
- Regardless of production mechanism (human or AI) work batches above 300 LoC must follow a pull-request path with evaluation by another human on the team
- Branching should be demonstrated and used over the project process but approach can vary as you go
- Versioning using [SemVer](#) must be employed
- A CI/CD pipeline must be built using GitHub Actions
- Deployment can happen to GitHub pages, Cloudflare, or as a downloadable asset, depending on the form.
- Testing with unit and e2e must be demonstrated, and not be applied only at the conclusion of the project. Significant early efforts with testing should be verifiable in the repository. Testing approaches may vary in tooling.
- Code documentation must be maintained as you go along in the project. This also includes commenting using JSDocs
- Linting and quality checks should be performed both manually by developers and via a CI pipeline for software artifacts (this may go beyond JavaScript code)
- Repositories should use a .gitignore file
- Commit messages should be consistent and follow a format like [Conventional Commits](#)
- A changelog should be kept and may be generated manually, automatically, or some hybrid
- A technical documentation site should be found either in a Github Wiki or private web site for future maintainers

Technical Constraints

- You may use markdown, standards-based HTML, CSS without a framework, vanilla JavaScript without a framework, media assets such as audio, video, image, fonts, or PDF files, JSON files, and any .txt or configuration files needed
- Any server-side based technologies must work on Cloudflare or Github pages only
- All major technical decisions must be captured as Architectural Decision Records (ADRs) in [MADR](#) format
- Dependencies can only be added with agreement from your teaching assistant (which implies proper justification)
- The form of what you build may include a traditional web application, progressive web application (Web app), wrapped mobile app (CapacitorJS), or a wrapped desktop app (ElectronJS). Chrome Extensions, VS Code extensions, and Slack extensions also have been seen and on occasion, REST endpoints for outside consumption make sense. Forms beyond any mentioned here must be given clearance.
- All projects must have a website to document what is produced for end users

Project Topic and Process

You will be given a topic as a prompt for you to clarify and design. Your TA mentor is your customer, and they work for the Prof who has driven the ideas. The topic itself will have many design decisions and open-ended aspects. You should do research and employ user-centered design throughout the project. This should include, minimally, a design brief, user personas, user stories, and wireframes. These artifacts should precede any non-exploratory work produced in the repository.

You will run a one-week sprint, Sunday to Sunday, for orienting, which is light work-wise due to the midterm. Then, a one-week design and prototyping sprint with initial exploration. Your TA will gate you on these two sprints and, if you do not perform them before heavy coding, you will be forced to repeat it the week after.

Your sprints will continue after, with a review break sometime in week 9. The review break will involve another team in your cohort that will use your software, look at your code, evaluate it, and provide product and code feedback. You will do the same. Interaction, or even software evaluation, should happen once from the Prof before the end of week 10, and you are expected to interact with the TA weekly.

Your Project Prompt

Title: *Agent Issue Tracker (AIT)*

Description: Issue tracking comes in many shapes and sizes. From the basics of GitHub issues to performance-minded systems like Linear, to the battleship known as JIRA. In the world of AI doing much of the work we want to revisit issue trackers and build one of our own from the ground up that fits into our AI workflow.

Can an agent read our issues? Can we track our tokens, budget, and time? Are there versions for we humans and some better suited for the agent? We probably need less reporting and less friction so we can plan at AI-level speeds. It's just a CRUD (Create, Read, Update, and Delete) application for sure, but it's the domain details that will make this perennial app challenging, especially if it has to integrate with other systems in the software engineering process.

Final Advice

Hi all - - Helena here! For the CSE 110 final project, each TA has a prompt tailored for their teams. I selected this project because I think this will be a great opportunity for you all.

Here are three important things to note:

(1) Build Something Resume-Worthy

I want you to leave this class with something you can proudly add to your resume.

When I took this class as an undergrad, my team built an application that I was really proud of. I mentioned it in almost every interview. Employers enjoyed hearing about it. One hiring manager at Slack even checked my GitHub activity and asked me detailed questions about the project.

The takeaway is that a class project can become one of your best interview stories if you put detailed thought and effort into it. Think back to when you first joined this class. What did you hope to gain?

Approach this as if you are building a real product, not just completing a homework assignment. This project really lets you showcase:

1. *CRUD + Domain Knowledge*

- ○ Model your domain knowledge on Agents cleanly and extensibly (via user input, creative design, LLM workflows, integrations with other software).

2. *Integration-Heavy Architecture*

- ○ Demonstrate how you integrate multiple APIs and services (LLM APIs, GitHub, Slack, etc.).

3. *AI-Enhanced User Experience*

- ○ Showcase how AI improves the product through features, intelligent suggestions, etc.

(2) **Shifting Focus (VERY IMPORTANT)**

This project will not be like the warm-ups where you were given step-by-step instructions. This spec is your “instructions”. Hence, for this first sprint, focus on the fundamentals.

DO NOT Split Into Sub-Teams It will be tempting to divide into frontend/backend teams. Do not do this in Week 5. For the love of your grade: **plan, plan, plan**

In the grad version of this class, my team and I split into separate tasks too early, leading to confusion and isolated work. We lacked a shared understanding and much of our work/code was wasted. Ultimately, your Week 5 goal should be *research + scope*

As a team, carefully review this entire document to. . .

Plan:

- Who will use your app?
- What are the user stories and personas?
- Functional requirements
- Non-functional requirements (“-ilities” from class)

Create:

- Document what you want the app to accomplish
- Shared Miro board or Google Doc (later posted on GitHub rep)
- Scope of future sprints

(3) **Align as a Team**

1. After reviewing the project, discuss **how much time you can realistically dedicate to it**

- This will give you all a better sense of the scope of your project.

2. Add Retrium to your team meeting this week: <https://app.retrium.com/>

- In your first meeting, use it to reflect on how your team handled the warm-up assignments. This will help you all decide how you want to structure your team moving forward. You will continue using Retrium to guide your sprint retrospectives.

I've intentionally left many details open as the "customer". It might help to start a shared Google Doc with your team to keep track of any questions as they come up.

Feel free to share your questions with your leads so we can talk about them in our 1:1s this week. Everyone is welcome to join those weekly meetings. You can also set up a Slack channel in your workspace called #ta-qa, or message me directly.

Good luck!